RallyIQ：本地化羽毛球视频分析系统的算法与架构

Nov 7, 2026 · @sam

摘要

RallyIQ 是一个完全在本机运行、免费开源的羽毛球视频分析系统：输入一段单机位比赛或训练视频，输出球场坐标、球员骨架与身份、羽毛球轨迹、回合与击球、击球类型及多层描述、打法风格、脚步和一份可分享的赛后报告。系统采用“规则提出候选 + 小模型打分 + 人工复核闭环”的混合架构：深度模型只用于像素级感知（TrackNet 式多帧热力图网络检测羽毛球、RTMO 估计人体姿态），其余判断由可解释的几何规则和导出为 JSON 的轻量模型（MLP、梯度提升树）完成，运行时只依赖 numpy，不需要付费 API 或云服务。

在公开数据集上的实测结果：羽毛球检测在 TrackNet 测试集上精确率 95.2%、召回 94.7%；修复方向翻转后，球场标定在 ShuttleSet 6 场留出比赛上 89.4% 的标注角点误差 ≤ 10 px（平均 16.5 px）；击球检测（±3 帧、区分近远方）在 14 段留出片段上 F1 从规则基线的 56.4% 提升到 74.6%，击球方判定正确率 95.4%；12 类击球类型在 6 场留出比赛上准确率 63.0%（规则基线 45.7%）。本文描述系统架构、各模块算法与设计取舍、评测口径和结果，并如实列出局限：精度目前只在 BWF 转播单打上测过，业余机位、双打和打法风格尚缺标注数据验证。

1 引言

业余球员和教练最容易拿到的是一部手机或运动相机拍的单机位视频，而专业分析系统通常依赖多机位、人工标注或付费云服务。RallyIQ 的目标是在一台普通电脑（主要是 Apple Silicon Mac）上，把这样的视频变成可信、可修正的技战术数据。

设计原则：

- **本地、免费、开源**：视频和结果不出本机；模型权重为 MIT / Apache-2.0；AI 教练默认走本机 Ollama + Qwen，没有时退回离线规则模板。
- **降级而不拒绝**：缺任何模型或某一步失败，对应输出为空或“不可判定”，其余阶段照常运行。
- **每个输出都带状态和理由**：`ok` / `needs_review` / `undeterminable` 加稳定的原因码，算不出就是 `null`，不填默认值，不伪造轨迹。
- **人工修正只追加不覆盖**：原始预测保留，修正层叠在上面；修正自动成为难例，用于后续微调。
- **运行时轻量**：只有羽毛球检测需要 PyTorch，姿态走 ONNX Runtime（Mac 上 CoreML）；击球打分、球种分类等学习模型都导出为 JSON，用 numpy 计算，无 scikit-learn、无 pickle。

本文的主要贡献：

1. 一条完整的单目羽毛球分析流水线，各阶段之间只通过带版本的 JSON 契约（pydantic 模型 + JSON Schema）通信。
2. 基于消失点分组与针孔相机约束的自动球场标定，支持机位移动跟踪和多场地画面中的目标场地选择。
3. “候选 + 学习打分 + 交替约束”的击球检测与击球方归属，在留出比赛上 F1 提升 18 个百分点。
4. 以可测证据为特征、导出为 JSON 树的 HistGB 球种分类器，以及叠加其上的多层击球描述、打法风格和脚步分析。
5. 一个“只有变好才替换”的羽毛球模型持续微调闭环（`shuttle_loop`）。
6. 一套按比赛分组、严格留出的评测口径，并保留了失败尝试的记录。

2 系统架构

系统分三层：感知层用深度模型把像素变成球场、人和球的坐标；理解层用规则和 JSON 小模型把坐标变成回合、击球和球种；分析层把修正后的事件变成统计、风格、脚步和报告。人工修正回流成为难例，驱动羽毛球模型持续微调。



RallyIQ 系统架构 · 三层处理 + 人工修正闭环

各阶段在 `pipeline/registry.py` 中注册为 `run(ctx) -> 契约模型`，按需导入，所以 API 进程从不加载 torch；羽毛球阶段可以与球员阶段并行预取。阶段之间只传 pydantic 契约（`contracts/`，附 JSON Schema 和示例），任何一个模块都可以单独替换。缺少模型权重时对应阶段返回空结果，其余阶段照常运行。

阶段	输入	输出契约	运行依赖
probe / 质量评估	视频	帧率、分辨率、`VideoQuality`	FFmpeg
calibration	视频帧	`CourtCalibration`（分段单应、置信度、其他场地）	OpenCV

阶段	输入	输出契约	运行依赖
players	帧 + 标定	`PlayerTracks`（骨架、身份、球场坐标、换边）	ONNX Runtime
shuttle	帧 + 标定	`ShuttleTrack`（检测点 / 补点 / 缺口）	PyTorch
events	球 + 球员 + 标定	`EventsResult`（回合、击球、击球方、原因码）	numpy
strokes	事件 + 球 + 球员	`StrokesResult`（类型、候选、证据、多层描述）	numpy
report	修正后的事件	`MatchStats`、`CoachReport`、HTML	numpy（可选 Ollama）

3 方法

全文统一球场坐标系：原点在离相机较近一端的左角，x 横向 0–6.10 m，y 纵向 0–13.40 m，网在 y = 6.70 m。所有阈值按毫秒、米、画面对角线比例或“球员身高”设定，所以 25/30/60/120 fps、720p–4K 用同一套参数。

3.1 球场标定

目标是求图像像素与标准球场（13.40 × 6.10 m）之间的单应矩阵 H，只依赖 numpy 和 OpenCV，不需要权重。单帧流程：

1. **场线证据**：对最亮通道做形态学顶帽变换，只保留比核细的亮结构（任意地胶颜色上的白线/黄线），得到线掩码和距离图。
2. **直线与分组**：Hough 线段合并成直线，按**消失点**而不是角度分成“平行底线”和“平行边线”两组，因此低机位、强透视、侧面机位都能分对。
3. **假设生成**：每组取两条线、模型取两条对应线，四个交点闭式求 H，批量计算。
4. **物理可行性筛选**：四角在相机前方、投影为凸四边形；并检查针孔相机约束：主点在画面中心附近时，地面两轴在 $K^{-1}H$ 中应正交且等长，在焦距范围内搜索最小违背。这一步去掉“整块球场压扁在一条亮线上”的退化解。
5. **打分**：投影全部场线采样点，得分 = 命中的不同像素格数 − 0.5 × 未命中格数；画面里不足 1 像素宽的远端线不计。
6. **精修**：沿投影法线找线中心，RANSAC 重拟合，迭代收紧容差。
7. **对称与朝向**：场线有四重对称，统一选“相机在地面上方、y = 0 靠近相机”的解；跟踪时选与上一段最接近的标注，保证切镜头后远近不翻转（PR #37 修复的就是这个问题）。

置信度由四项相乘：场线命中率（扣除随机放置也能达到的命中率）、独立核验的场线交点数、是否存在同样好的另一种线对应（歧义）、球场占画面比例。≥ 0.6 为 `ok`，≥ 0.3 为 `needs_review`，否则 `undeterminable`（特写和回放应落在这里）。

视频跟踪：`CourtTracker` 每 500 ms 检查一帧。场线仍对齐时只做精修，累积位移超过对角线 0.6% 开新段；不再对齐时用 ORB 特征估计相机运动，把上一段的 H 搬过来作候选后重标；直方图突变（切镜头）则从头标定，找不到球场就输出 `undeterminable` 段并持续重试。事件阶段会忽略标定占比低于 0.25 的场外镜头，避免在观众和回放画面上报击球。

目标场地选择：场线只能选出“最像球场”的场地，在体育馆多场地画面中可能选中没人打球的邻场（DJI_0009 就是例子）。`court_choice` 在每个机位段抽 12 帧跑姿态，统计每块候选场地“两个半场都有人”的程度，必要时换成更像正在比赛的场地；都不像时标为不可判定，提示用户点选四角。手动点选的结果会吸附到实际场线上。

3.2 羽毛球检测与跟踪

网络：TrackNetV3 式的 U-Net 热力图网络（`HeatmapNet`），输入是 8 帧连续 RGB 帧加上当前机位段的背景中值图（抑制静止亮点），缩放到 512 × 288，输出每帧一张热力图。窗口逐帧滑动，每帧的热力图是所有包含它的窗口的加权平均（时间集成）。预训练权重来自 TrackNetV3（MIT）。高帧率视频（如 120 fps）拆成约 30 fps 的交错子序列分别推理，让网络看到的帧间运动接近训练条件。

候选：每帧保留最多 3 个热力图峰值，阈值 0.2（常规做法是 0.5 且只取最大块），亚像素质心。远端小球的峰值往往只有 0.2–0.4，交给跟踪器靠时间连续性判断。

跟踪（只依赖 numpy）：按匀速预测和随速度变化的门限逐帧关联成小段；按总分从高到低挑选小段，每帧只保留一个点；弱候选只有连成 ≥ 3 帧的一致小段才保留；去掉孤立跳点，但保留击球时的方向突变；≤ 0.3 s 的短缺口用二次曲线补点（拟合不好时只对 ≤ 0.1 s 的缺口线性补），补点标记 `interpolated` 且置信度更低；长缺口保持为缺口，不伪造轨迹。所有候选（包括被拒绝的）都写入 `shuttle_candidates.json`，供复核和积累难例。

多场地锁定（`court_lock`）：把目标场地上方的“空中区域”（场地中部最高 10 m，底线和边线外更低）投影到画面，按整条轨迹计算归属权重，平均权重 < 0.5 的轨迹整条丢掉。合成评测中，左右相邻场地的串场率从 57–73% 降到 0%，召回不降；前后相邻是单目视角的难点，串场率从 95% 降到约 23%。

为什么不直接降低阈值：在 4 段真实片段上试过 0.1，跟踪覆盖提高（副场 50% → 66%，主转播 69% → 76%），但新增的低分点制造假的方向变化，高置信击球在每段都减少（如 ae26 27 → 21、court3 12 → 5）。结论是检出多了不等于准了，提高覆盖要靠微调而不是调阈值。

3.3 羽毛球模型持续微调（shuttle_loop）

预训练模型来自职业转播画面；在副场固定机位（高位俯拍，1080p 中球约 10–12 px，缩到 512 × 288 后只剩约 3 px）上覆盖只有约 50%。`scripts/shuttle_loop.py` 把“标一段 → 训一轮 → 比一次”变成一条命令：

1. **预标注 + 只修可疑帧**：标注工具以模型预测打底，`g` 只在低分检测、≤ 15 帧短缺口、长缺口两端的可疑帧之间跳转。只有 `reviewed` 的标注进入训练，避免模型学自己的错。
2. **切分**：每段按时间前 70% 训练、后 30% 验证；`--holdout` 让整段只做验证。
3. **训练**：从预训练权重开始，BatchNorm 冻结、学习率 1e-4、10 轮 × 300 步、batch 4，数据增强只有水平翻转和亮度/对比度抖动；按阈值 0.2 下的 F1 选最好的一轮（按 0.5 选曾选出一个在 0.2 下更差的轮次）。
4. **冠军制替换**：新模型和现役模型在**同一批**验证帧上比较，F1@0.2 至少高 0.01 且精确率下降不超过 0.03 才采用；可一键回退。
5. **标注核查**：每轮用最新模型过一遍所有已标注帧，列出模型与标注对不上的帧（误报 / 位置错 / 漏报），拼成 48 帧的核对图，在标注工具里逐帧确认。这解决了“预标注把模型漏检的可见球标成看不见”的问题。

这个闭环的核心是“只有变好才替换”：验证分数是逐帧检测，采用后还要在没参与训练的片段上核对击球结果。

3.4 球员姿态与身份追踪

姿态估计：RTMO-s（OpenMMLab，Apache-2.0，body7 训练，640 × 640 输入）的 ONNX 导出，用 onnxruntime 运行，输出人框和 COCO-17 关键点。远端球员在模型输入里低于 110 px 时，按单应裁出远端半场（含界外缓冲和头顶空间）放大再跑一遍（far-boost），并按落脚点距离去重，避免同一人被检成两个。

足点与过滤：足点取两踝中点并略微下移，经 H 映射到球场坐标。每条过滤规则只降低 `player_score` 并记录原因：足点在场外缓冲区（两侧 1 m、两端 2 m）之外、坐姿（裁判、教练）、身高与该位置期望像素高度不符、脚被画面底边截断。

单打 / 双打识别：缓存前 90 个处理帧的候选，“场内高分候选 ≥ 3 且某半场至少 2 人”的帧占比 ≥ 40% 判为双打。

身份追踪：单打固定槽位 P1/P2，双打 P1–P4（A = P1 + P3，B = P2 + P4）。每帧对所有分配方式一次性打分，代价由运动连续性（带速度门限）、上衣颜色相似度、候选分数和半场先验组成。单打里两个槽位不可能都在同一半场，这种矛盾的“半场记忆”会被忽略（PR #40，修复后 sx36_set2 的两名球员同时检出率 0% → 88%）。≤ 6 帧的漏检线性插值并标记 `interpolated`。

换边与身份校正：真实比赛只在局间换边。跟踪结束后按“身份段”整体校正：只有新半场持续 ≥ 15 秒（期间 ≥ 80% 有效帧同意）且距上次换边 ≥ 180 秒才算真换边，否则视为身份分反并整段对调。该规则上线后，4 段 BWF 片段的误报换边从 2–7 次降到 0。

3.5 回合切分、击球检测与击球方归属

这是全系统最难、也投入最多的一环。最终方案是“规则提出候选 + 学习模型打分 + 动态规划保证交替”的两段式结构，距离一律以最近球员的画面身高 H 为单位，远端球员按自己的尺度衡量。



击球检测流程 · 8 步，学习打分在中间

第一行全是规则，只负责“不漏”；第二行从右往左读，学习打分和交替约束负责“不错”。

回合：球可见且在运动时回合进行；球消失 1.2 s、或静止 0.6 s（落地）、或视频结束时回合结束；起点取球开始移动前的最后一个静止点（发球）。标定不可用的镜头段（特写、回放）被屏蔽。

候选：在每个轨迹点用最小二乘估计前后 130 ms 的入射/出射速度（遇遮挡可延长到 450 ms），转角 ≥ 35° 或加速比 ≥ 1.8 即为候选（单纯减速不算，空气阻力一直在减速）。在 280 ms 窗口内做非极大值抑制，按速度冲量 |v_out − v_in| 乘以球拍邻近度排序。

球拍邻近度：球到每名球员手腕和外推球拍头的距离，球拍头估计为

```
p_racket = p_wrist + k · (p_wrist − p_elbow),   k = 0.7
```

超过 0.9 H 视为无人击球（落地、高球顶点、假检测）。只用手腕、不外推球拍头的做法试过，F1 66.1% → 65.5%，更差。

学习打分（hit scorer）：原始规则的置信度是手工加权

```
conf = 0.35 · change + 0.35 · proximity + 0.15 · sharpness + 0.15 ·
track_quality
```

真实比赛里误报多。现在每个候选计算 27 个特征（转角、速度变化、冲量对数、离两名球员球拍的距离和差距、是否用了手腕、遮挡、插值、球置信度、离回合起止的时间、是否落地、来去方向和速度、击球点相对身体的高度等），由一个单隐层 MLP（或逻辑回归）给出“±3 帧内有标注击球”的概率。模型选择用按比赛分组的 5 折交叉验证，在逻辑回归、隐层 8/16 的小网络和三种正则强度中按对数损失选择；再在交叉验证预测上做端到端模拟（过滤 → 补发球 → 交替约束），选 F1 最高的 keep / ok / min_candidate 阈值组合。由于速度冲量峰值往往早于标注接触帧 1–2 帧，模型同时输出一个帧偏移（v1e 为 +1 帧）。模型存为 JSON，用 numpy 计算。

方位模型：另一个小模型根据球的来去方向和击球点相对身体的高度，判断击球来自近端还是远端；只有把握 ≥ 0.8 时才推翻“最近球员”的判断。

交替约束：一个小动态规划在每个回合里挑出最能解释“击球方交替”的候选子集：夹在对方两拍之间的弱候选可以保留，弱的重复候选被删除。双打中若最近的球员所在队刚击过球、而另一队有人几乎同样近，改判给另一队并标 `hitter_from_alternation`。

缺失击球方的补全（PR #62）：时间找对的击球里约 7% 没有击球方（多在回合开头，姿态帧会空 5–20 帧）。现在按“双方轮流击球”补上：选与前后已知击球不同一方的球员，姿态缺失时前后最多找 25 帧，这些击球仍进复核。

姿态序列补击球（可选，默认关闭）：球没被看到时，用球员约 0.5 s 的姿态序列（手腕高度和速度、手肘、前倾、下蹲）输入一个梯度提升树，补回发球和“同一方连击”之间漏掉的对方击球。

每个击球带稳定原因码（`hitter_ambiguous`、`shuttle_occluded_at_hit`、`low_track_quality`、`same_hitter_consecutive` 等 14 种），只有置信度 ≥ 阈值、击球方已知且没有原因码的才是 `ok`。

3.6 击球类型分类（HistGB）

类别：12 类，与 ShuttleSet 的 25 类（“未知”+ 上/下半场 × 12）去掉半场前缀后一致：发短球、发高远球、高远球、杀球、吊球、挑球、放网、勾对角、推球、扑球、平抽、挡网。

特征 = 可测证据。分类器不直接看像素，而是看前面各阶段已经测出的证据，测不到就是 NaN，不填猜测值：

特征组	内容
基础证据	击球点高度 `contact_h`（以击球者“脚踝→鼻子”长度为 1）、击球者/接球者离网距离（m）、场地速度（m/s）、飞行时间、弧高 `rise`、是否飞出画面上沿、横向位移
回合上下文（v2）	来球（上一拍的飞行时间、弧高、对手击球高度和站位）、回球（下一拍的击球高度和飞行时间）
方向（v2）	击球方和接球方的左右位置、是否斜线
姿态与出球（v2）	最高手腕和手肘相对头部的高度、手腕离身体的距离、击球前后 200 ms 的球速（以身高为单位）

模型：scikit-learn 的 `HistGradientBoostingClassifier`（100 棵树 × 12 类），原生支持缺失值。训练后用 `gbt.export_hgb` 把每棵树导出为平行数组（特征、阈值、NaN 去向、左右子节点、叶值），运行时用 numpy 计算 baseline + 各树之和再做 softmax。模型文件约 1 MB，随仓

库发布（`strokes_v2.json`），不依赖 scikit-learn 版本，也没有 pickle 的安全问题。训练数据：ShuttleSet 19 场比赛的 85 段片段、1270 拍。

复核策略（与后端无关）：可测证据不足 30% 为 `undeterminable`；`ok` 需要置信度 ≥ 0.6、第一名比第二名高 ≥ 0.2、证据 ≥ 80% 可测、且击球本身是 `ok`。没有权重时退回 `rules_v1`：把每种类型写成证据的模糊“与”（0–1 斜坡），测不到的证据按 0.5 处理。

试过但未采用：加入击球前后的挥拍姿态序列（测试 61–63%）、用前后拍预测类型做第二阶段（60.9%），都不如 v2 的 63–65%。BST（骨架 + 羽毛球 Transformer）的数据预处理已接入，但作者权重没有单独许可证，只作本地研究对比。

3.7 多层击球描述（开发分支）

教练不会只说“杀球”，而是“正手直线杀球，击球点较高，出球快，属于主动进攻”。`detail.py` 在类型之上叠加六层描述，每层都标明来源（`model` / `measured` / `rule`），前端对规则推断的层用虚线标签，AI 教练也被告知这些只作参考：

层	取值	来源
大类 / 细分	10 个大类；勾对角、发短球、发高远球	由类型合并
正反手	正手 / 反手 / 头顶	HistGB 模型
击球点	网上 / 网下	HistGB 模型
方向	直线 / 斜线	HistGB 模型
快慢	快 / 中 / 慢	轨迹实测，与同类球的 33/67 分位速度比（如杀球 18.3 / 22.6 m/s）
主动 / 被动、意图	进攻 / 相持 / 被动；进攻 / 调动 / 控制 / 防守 / 过渡	规则推断（类型 + 击球点 + 上一拍）

三个模型头用 ShuttleSet 的 `backhand`、`aroundhead`、`hit_height` 列和击球点/落点坐标训练（19 场 1310 拍），输入是 v2 特征 + 击球前后的姿态窗口 + 球相对身体的左右位置 + 从高点击球推出的持拍手。**明确不输出**的：点杀、劈杀、劈吊、搓球、平快发球、偷后场发球和“欺骗”意图——数据里没有标签，或者单目视频看不出拍面和切削（杀球 vs 点扣在留出比赛上只有 54%，等于瞎猜）。

3.8 打法风格（开发分支）

有标注的风格数据几乎不存在，所以这里不训练模型，而是“可测特征 + 原型匹配”（`report/style.py`）：

1. **逐拍攻防状态**：按来球判断击球者处于主动（对手挑球、或被压制时打的高远球）、被动（对手杀球/扑球）或中立。单个 30 fps 机位看不到击球点高度和重心，所以用来球类型代替。
2. **特征**：约 25 个，包括击球构成（杀球、网前、拉吊、平抽推压占比）、主动转化率、连续进攻拍数、前两拍抢攻、拉吊突击、防守反击、被动顶住率、节奏（飞行时间中位数和变异系数）、对手移动距离、四角调动、直线比例、同位置落点多样性。每个特征有以精英单打统计文献为锚的参考区间 [lo, hi]（如杀球占比 5–20%）。
3. **原型匹配**：单打 8 种原型（强攻压制、速度抢攻、拉吊突击、控制组织、防守反击、多拍消耗、技术欺骗、节奏变化），双打 5 种体系。每种原型是一组带方向和权重的特征，得分 0–100。
4. **小样本收缩与拒绝判断**：每个特征按样本量向中性收缩，少于 20 拍为“证据不足”，最高分 < 60 为“均衡型”；单打相邻两拍交替比例 < 80% 时降为低可信，< 60%（接近随机）时不给结论。

```
match = 0.5 + (m − 0.5) · n / (n + 10)          ▷ n 为该特征的样本量
```

另外统计六种单打战术套路（四角调动、网前逼起球、拉吊突击、防守转攻、重复落点陷阱、节奏破坏）的次数、所在回合胜率和可回看的视频时间，以及“对手打某种球后此人最常见的回球”。

3.9 脚步分析（开发分支）

只用姿态模型的两个脚踝关键点，纯 numpy，3 分钟 30 fps 视频约 0.3 秒：

输出	方法
触地	脚踝下移约 8 cm 得到着地点；按“每米像素数”换算脚的图像速度，< 0.55 m/s 着地、> 1.0 m/s 离地（滞回）。用图像速度而非球场速度，因为低机位下远端纵向 1 米只有几个像素
步	同一只脚相邻两次着地位移 ≥ 0.18 m（远端按 3 倍定位误差放宽）
启动步	双脚去趋势后同时上升再同时下落；落地相对对手击球在 −0.1～+0.3 s 为及时
起动	对手击球后，双脚中点朝自己下一拍击球位置的速度持续 0.1 s 超过 1 m/s 的时刻
步法 / 弓步 / 起跳	并步、交叉步、跑动；两脚相距 ≥ 1 m 且最后一步 ≥ 0.5 m 为弓步；双脚同时离地 0.2–0.9 s 且抬高 ≥ 12 cm 为起跳

输出	方法
回位	击球后多久回到本方中心（中线、离底线 3.6 m，半径 1 m）；对手下一拍出手时离中心多远

每名球员给出着地抖动 `jitter_m`：近端一般 0.03–0.05 m，远端低机位约 0.1 m，此时结果标为“只作参考”。

3.10 赛后统计、逐拍建议与比赛报告

统计：全部基于人工修正后的事件。每个指标是一个 `Metric`（数值、单位、样本量 n、状态、说明），算不出就是 null。移动只统计回合内非插值的足点，5 帧中值去抖，> 8 m/s 的跳变剔除；站位按“本方半场”换算，换边后仍可比；标定不是 `ok` 时米制指标降为待复核。

逐拍建议（`moments.py`，不调用模型）：取击球时双方的球场站位、对手为接这拍移动的距离、对手的回球和结果，用 5 条规则找出该换打法的球（如“对手已在后场你还挑后场”“对手移动不到 1 米就接到”），并给出离对手最远且从击球位置打得到的目标区域。

AI 教练与防编造：默认用本机 Ollama + Qwen2.5-7B，不可用时退回离线模板。模型只写观点和建议文字，时间、回合、站位永远取自数据。`grounding.py` 对模型输出做三条校验：引用不存在的指标被去掉；观点可信度不高于所引用指标中最低的一个；文中数字必须能在引用数据里找到，否则标记“含未核实数字”。训练建议从 `knowledge/coaching` 的教学笔记中检索，只能引用本次检索到的笔记，链接和标题从笔记文件取。

可分享报告（`html.py`）：一场比赛生成一个独立 HTML 文件，样式和 SVG 图表内嵌，没有脚本、不联网、不含视频，包含概览、教练点评、回合、击球类型、多层描述、打法风格、脚步、场地热区和逐拍建议；所有字段按可选读取，缺哪一块就不显示。

3.11 核心算法（伪代码）

下面八个算法对应代码中的实际实现，文件名标在每个算法标题里。符号：H 为单应矩阵（球场米 → 像素），H_p 为最近球员的画面身高，t 为毫秒时间。

算法 1：单帧球场标定（`calibration/calibrate.py`、`homography.py`）

```
输入：帧 I；标准球场模型 M（场线与交点，单位米）
输出：单应 H、置信度 conf、其他场地列表

mask ← TopHat(最亮通道(I)) 中比核细的亮结构；D ← 距离变换(mask)
L ← 合并(HoughLinesP(mask))
(V1, V2) ← 两个主消失点；L_1 ← 指向 V1 的线，L_2 ← 指向 V2 的线（两组都符合的线两组都放）
对 L_1 中任两条线 × L_2 中任两条线 × M 中对应的两条横线和两条纵线：
    H ← 由四个交点闭式求解（单位正方形 → 四边形）
    若 四角不在相机前方 或 投影非凸 或 太小：丢弃
    若 min_f ‖K_f⁻¹H 的两列不正交或不等长‖ 超限：丢弃          ▷ 针孔相机约束
    score(H) ← #命中像素格 − 0.5 · #未命中格                ▷ 投影宽度 < 1 px 的远端线不计
对得分最高的若干 H：重复 { 沿投影法线找线中心 → RANSAC 重拟合 H → 收紧容差 }
不重叠且同样好的解 = 相邻场地；取最靠近画面中心的一块，其余放入“其他场地”
在四重对称解中选“相机在地面上方、y=0 靠近相机”的一个（跟踪时选与上一段最接近的）
conf ← 命中率超出随机基线 × 核验交点得分 × (1 − 歧义) × 球场占画面比例
```

算法 2：羽毛球小段跟踪（`shuttle/tracker.py`）

```
输入：每帧最多 3 个热力图峰值候选 C_f（分数 ≥ 0.2）
输出：每帧至多一个点（detected / interpolated）与缺口列表

关联：逐帧把候选接到现有小段末端——用匀速模型预测位置，门限半径随速度增大；
    接不上就开新小段
多场地时：每个小段按“目标场地上方空中区域”的平均权重过滤（< 0.5 整段丢弃）
选择：按小段总分从高到低选，和已选小段在时间上冲突的帧让给高分者；
    弱候选组成的小段必须 ≥ 3 帧且运动一致才保留
去跳点：离前后邻居都很远的孤立点删除，但前后方向一致的转折（击球）保留
补点：对长度 ≤ 0.3 s 的缺口拟合二次曲线；拟合差时只对 ≤ 0.1 s 的缺口线性插值；
    补点置信度低于检测点，不补到画面外
其余缺口保留，并标注 out_of_frame（球飞出画面）或 not_detected
```

算法 3：单个回合内的击球检测（`events/hits.py: detect_hits`）

```
输入：回合内的球轨迹 P、姿态索引、打分模型 S（可选）
输出：按时间排序的击球列表

对每个轨迹点 i：
    v_in, v_out ← 前、后 130 ms 的最小二乘速度（遇遮挡最多延长到 450 ms）
    若 转角(v_in, v_out) ≥ 35° 或 |v_out| / |v_in| ≥ 1.8：加入候选 c_i
    c_i.prox ← 球到每名球员手腕与球拍头的距离（以 H_p 为单位）
非极大值抑制：按 merit = |v_out − v_in| · (0.5 + 邻近分) 降序取候选，
    丢弃与已取候选相距 < 280 ms 的
对每个保留的候选 c：
    rule_conf, 原因码 ← 规则判断（离球拍 > 0.9 H_p、落地、遮挡、轨迹质量……）
    若有 S：x ← 27 维特征；c.conf ← S(x)；若方位模型把握 ≥ 0.8 则改判近/远端
    否则：c.conf ← 0.35·change + 0.35·proximity + 0.15·sharpness +
0.15·quality
    保留 c.conf ≥ min_candidate_conf 的候选
若第一拍距回合起点 > 280 ms：在发球者球拍处推断一个发球候选
返回 ResolveSequence(候选)                          ▷ 算法 4–5
```

算法 4：交替约束动态规划（`events/hits.py: _alternating_subset`）

单打里相邻两拍必须是不同球员，双打里必须是不同队伍。给每个候选一个权重 w_i = conf_i − τ_keep（τ_keep = 0.45），在“方位交替”的约束下求权重和最大的子序列：

$$w_i = \text{conf}_i - \tau_{keep}$$
$$B(i) = w_i + \max\left(0, \max\{ B(j) : j < i, \text{side}(j) \neq \text{side}(i) \}\right)$$

```
对 i = 1..n：B[i] ← w_i；prev[i] ← 空
    对 j < i 且 side(j) ≠ side(i)：若 B[j] + w_i > B[i]：B[i] ← B[j] + w_i；
prev[i] ← j
从 argmax B 沿 prev 回溯得到集合 chosen
返回 { c_i | i ∈ chosen 或 conf_i ≥ τ_ok }      ▷ 很强但不合交替的候选保留并标记
```

效果：一个弱候选如果夹在对方两个强击球之间（交替需要它），会被保留；一个弱的重复候选会破坏交替，被删除。复杂度 $O(n^2)$，一个回合的候选通常不到 50 个。

算法 5：击球方改判与补全（`events/hits.py: _reattribute`、`_fill_hitters`）

```
1  改判：对相邻两拍 (a, b) 若 side(a) = side(b)：
2      k ← b 的邻近列表里最近的另一方球员
3      若 邻近分(b 的最近者) − 邻近分(k) < 2 · hitter_margin：把 b 改判给 k，标
hitter_from_alternation
4  补全：对击球方未知的击球 c：
5      pool ← c 时刻的球员；为空则在前后 25 帧内找姿态
6      avoid ← { 前一个已知击球的一方，后一个已知击球的一方 }
7      若 pool 里没有不在 avoid 的人：avoid ← { 前一个已知击球的一方 }
8      c.击球方 ← pool 中不在 avoid 、离球最近的人；标 hitter_from_sequence，进复核
9  最后：仍然同一方连击的两拍都标 same_hitter_consecutive
```

算法 6：羽毛球模型冠军制微调（`scripts/shuttle_loop.py`）

```
输入：已复核标注片段集 R、现役模型 θ*、预训练权重 θ₀、留出片段集 O

1  若 R 与上一轮相同且未加 --force：结束
2  对每个 r ∈ R：若 r ∈ O 则整段入验证集，否则前 70% 帧入训练集、后 30% 入验证集
3  θ ← θ₀；冻结 BatchNorm；学习率 1e-4
4  对 epoch = 1..10：训 300 步（batch 4，水平翻转 + 亮度对比度抖动）；记录验证集
F1@0.2
5  θ^ ← F1@0.2 最高的一轮
6  在同一批验证帧上评估 θ*
7  若 F1(θ^) − F1(θ*) ≥ 0.01 且 P(θ*) − P(θ^) ≤ 0.03：θ* ← θ^（旧模型存档以便回退）
8  用最后一轮模型遍历所有已标注帧，输出误报 / 位置错 / 漏报帧供人工核对
```

算法 7：打法原型匹配（`report/style.py: match_prototype`）

每种原型 p 是一组带符号的特征权重 w_k（负号表示该特征偏低才典型）。特征值 v_k 先按参考区间 [lo_k, hi_k] 归一化，再按样本量 n_k 向 0.5 收缩：

```
x_k = clip( (v_k − lo_k) / (hi_k − lo_k), 0, 1 )
x̃_k = x_k          若 w_k > 0
      1 − x_k      若 w_k < 0
m_k = 0.5 + (x̃_k − 0.5) · n_k / (n_k + 10)
```

```
score(p) = 100 · [ Σ_{k∈M} |w_k|·m_k + 0.5 · Σ_{k∉M} |w_k| ] / Σ_k |w_k|
```

其中 M 是可测特征的集合（n_k ≥ 5）。测不到的特征按“中性”0.5 计，所以一个原型不会只靠容易测的特征拿高分；可测权重不足 50% 时不打分。最高分 < 60 判为“均衡型”，第二名与第一名相差 ≤ 5 分时一起列出。

算法 8：脚步触地、启动步与起动（`footwork/`，开发分支）

```
输入：每帧左右脚踝关键点、单应 H、修正后的击球列表

着地点 ← 脚踝下移约 8 cm；若左右脚与上一帧交叉更近，则交换左右
每只脚：v_img ← 图像速度 / 当地“每米像素数”（m/s）
状态机（滞回）：v_img < 0.55 → 着地；v_img > 1.0 → 离地
步 ← 同一只脚相邻两次着地的球场位移 ≥ max(0.18 m, 3·定位误差)
启动步 ← 双脚去慢速趋势后同时上升再同时下落；
        落地时刻 − 对手击球时刻 ∈ [−0.1, +0.3] s → 及时；更早 → 偏早；更晚 → 偏晚；
        ±0.45 s 内没有 → 没有
起动 ← 对手击球后，双脚中点朝“自己下一拍击球位置”的速度连续 0.1 s > 1 m/s 的首个时刻
回位 ← 击球后首次进入本方中心（中线、离底线 3.6 m、半径 1 m）的时间；
        对手下一拍出手时离中心的距离
jitter ← 着地期间脚位置的抖动（m）；远端 jitter ≈ 0.1 m 时结果标“只作参考”
```

4 实验与评估

主要结论：羽毛球检测和球场标定在转播画面上已经可用；击球检测从 56.4% 提升到 74.6% F1，但离产品门槛（F1 ≥ 0.90）仍有距离；击球类型 63%，推球/挑球、平抽/挡网最难分。以下所有精度都来自人工标注，并明确标出是留出测试还是交叉验证。

4.1 数据与评测口径

数据	内容	用途
ShuttleSet（KDD 2023，MIT 标注）	44 场 BWF 转播单打，逐拍击球帧、击球方、类型、正反手、击球高度、球场角点	38 场训练；第 1、8、15、22、29、36 场整场留出测试
TrackNet 测试集	3 场 29 个回合，逐帧球位置	羽毛球检测
BWF 评测片段	19 段，覆盖五个单项、主转播 / 副场固定机位 / 集锦	无标注覆盖率统计、副场微调
自拍 DJI_0009	业余单打对练，底线后低机位，1080p30	案例分析（无标注）

口径：击球以 ±3 帧为命中，并要求近端/远端判对（“分方”）；回合以 IoU ≥ 0.5 为命中；标定以标注角点误差 ≤ 10 px（换算到 1080p）为成功。所有模型选择和阈值调整只在训练场次上做按比赛分组的 5 折交叉验证，同一场比赛不会同时出现在训练和验证里；6 场留出比赛不参与任何调参。

4.2 羽毛球检测与球场标定

羽毛球检测在 TrackNet 测试集上精确率 95.2%、召回 94.7%。这是上限：权重就是在该数据集的训练集上训练的，测试集是同类转播画面。在副场固定机位的无标注统计中，跟踪覆盖只有 50–66%，这是引入 `shuttle_loop` 的原因。

球场标定在 ShuttleSet 6 场、170 个标注点上的结果（PR #37 修复了“片段中途角点顺序整段翻转”的问题）：

指标	修复前	修复后
标定成功率（≤ 10 px）	61.2%	**89.4%**
平均角点误差	281.9 px	**16.5 px**
中位角点误差	9.6 px	7.2 px

翻转分为三种：旋转 180°、左右镜像、近远镜像，都是“从某一刻起整段翻转”而不是抖动，说明几何是对的、标签方向错了。修复方法是重新标定时保持与上一段一致的方向。修复后 4/6 场 100% 在 10 px 内；重投影误差在 BWF 片段上为 1.4–2.7 px。

4.3 回合与击球检测

14 段留出片段（6 场比赛，模型从未见过）上各版本的对比：

版本	训练数据	精确率	召回	击球 F1	不分方 F1
原规则（手工加权）	—	50.8%	63.4%	56.4%	66.4%
v1c：学习打分 + 方位模型	22 段	67.5%	60.7%	63.9%	76.2%
v1e：+ 击球帧 +1 帧校正	35 段	81.5%	61.9%	70.4%	78.8%
v2a	49 段	77.5%	64.2%	70.2%	81.3%
v1e + 缺失击球方补全（PR #62）	35 段	—	—	**74.6%**	—

- **提升主要来自误报减少**：精确率从 50.8% 升到 81.5%，召回基本不变。规则候选只覆盖 81% 的标注击球，剩下 19% 是球没检到或候选被合并，打分模型补不回来，这是召回的上限。
- **击球方判定**：时间找对的击球里，近远方判对的比例从 89.7% 升到 95.4%（规则 87.0% → 92.3%）。关掉“交替改判”会降到 92.6%。
- **场外镜头屏蔽**：忽略标定不可用的镜头段后，回合 F1 从 55.0% 升到 82.1%（规则击球 F1 56.4% → 62.2%），257 个标注击球里只有 6 个落在被屏蔽的帧里。此前的诊断显示，127 个误报中有 80 个在标注回合之外（捡球、走动、回放）。
- **姿态序列补击球**（可选）：留出比赛上 F1 69.5% → 71.9%，训练比赛折外 70.2% → 71.7%，收益主要来自同方连击之间漏掉的对方击球。

- **合成基准**：干净条件 F1 0.99，强噪声（4 px、15% 丢失、3% 假检测）0.87。只说明规则按设计工作，不代表真实视频。

失败尝试：参数搜索（v1d）在测试集上无提升；“同方 0.8 秒内连击改判”和“缩小近端大个子的判定范围”让序列看起来更合理（同方连击 23.1% → 16.0%），但 F1 几乎不变，默认关闭；用姿态模型改判已检出击球的方位会降低 F1。

4.4 击球类型与多层属性

6 场留出比赛（184 拍，只统计时间上对得上标注的击球）：

方法	测试准确率	测试宏 F1	训练集按比赛交叉验证
规则 `rules_v1`	45.7%	—	42.4%
HistGB，基础特征	56.0%	0.43	64.5%
HistGB，特征 v2	**63.6%**	**0.51**	67.4%
随仓库发布的 `gbt_v2`（85 段训练，经过完整复核策略）	63.0%（模型本身 65.2%）	—	68.0%

测试只有 184 拍，±3.5 个百分点以内的差别不可靠。去掉任何一组特征（方向、来球、回球、出球速度）测试准确率都会下降。交叉验证中各类 F1：发短球 0.84、杀球 0.81、高远球 0.79、挑球 0.77、放网 0.72、挡网 0.65、吊球 0.62、勾对角 0.55、推球 0.36、扑球 0.36、平抽 0.17；发高远球只有 3 个样本。主要混淆是推球→挑球（48 次）、挡网↔放网、平抽→挡网。

多层属性模型在 6 场留出比赛（191 拍）上：正反手 75.4%（平衡准确率 72.0%，全猜多数类 49.2%），击球点高低 83.8%（82.1%，60.2%），直线/斜线 93.5%（92.8%，61.4%）。头顶球召回最低（0.54）。

4.5 运行效率

Apple M4 Pro（14 核，48 GB）上 90 秒 1080p30 片段的全流程约 7.5–15 分钟。这些耗时是在机器负载很高时测的，只能粗略参考：羽毛球阶段在 MPS 上 5.8–19 fps；RTMO-s 单帧约 143 ms（CoreML 只接管了 656 个节点中的 195 个，实际没有加速）；事件阶段 < 0.5 s；脚步分析 3 分钟视频约 0.3 s。姿态是最大的端到端瓶颈。

4.6 案例：自拍业余视频 DJI_0009

3 分钟片段（手动标定四角，17 个回合、110 拍）暴露了从转播画面迁移到业余机位时的问题：

- **击球方交替只有 46%**（单打本应接近 100%），P1 77 拍、P2 33 拍，70 拍带 `same_hitter_consecutive`。打法风格因此拒绝给出结论；在加保护之前，P1 会被误判为“节奏变化型 81 分”，原因正是漏检让两拍间隔忽长忽短。

- **脚步**：近端着地抖动约 0.05 m（可用），远端约 0.11 m（只作参考）。逐帧对照视频，0:44 的弓步（两脚相距 1.2 m）和 2:25 的起跳杀球都能识别。P1 在对手击球时做启动步的比例为 33%，对照职业比赛约 60%；对手出手前回到中心 P1 74%、P2 33%。
- **球场选择**：自动标定选中了远处邻场的线，促成了 `court_choice` 的“靠球员分布反验场地”逻辑。

这个案例说明：下游的风格和脚步分析完全受限于击球与击球方的质量，系统选择在证据不足时如实拒绝判断，而不是输出看似合理的结论。

5 局限与未来工作

最大的局限是评测覆盖面：所有带标注的精度都来自 BWF 转播主机位单打，而用户真正会拍的是低机位、业余场馆、球在背景墙和灯前的画面。

局限	影响	计划
只在转播单打上测过精度	业余机位、副场、双打的精度未知	用 T7 硬盘上 85 个自拍视频（约 12.2 小时）分批标注；优先 withneric 场次，DJI_0009 固定为考试题
击球召回卡在约 62–66%	规则候选只覆盖 81% 的标注击球	提高球检测覆盖（shuttle_loop），姿态补击球，改进候选生成
击球方在业余视频上不可靠	DJI_0009 交替只有 46%，风格和脚步被拖累	同场次标注击球与击球方；改进球员跟踪丢人
球种分类 63%	推球/挑球、平抽/挡网、放网/勾对角易混	更多标注；评估在自有数据上重训的 BST
打法风格、意图、主动/被动没有标签可测	只能作教练式参考	与教练共同标注少量比赛做一致性评估
单目、无畸变校正	广角镜头远端误差变大；前后相邻场地串场率仍约 23%；镜像视频无法仅凭场线判断	加入畸变估计；用高中联赛的多场地画面测试锁定
脚步只有脚踝关键点、30 fps 下启动步仅 3–5 帧	踮脚会让着地点偏远；远端只作参考	用 60 fps 素材；可选 Halpe-26 / COCO-WholeBody 足部关键点
姿态是速度瓶颈	CoreML 只接管 30% 节点，没有加速	改导出格式或换更适合 ANE 的模型

局限	影响	计划
训练数据的许可	ShuttleSet 标注是 MIT，但视频归 BWF；由此训练的模型只用于个人研究	用自有授权素材重新训练后再公开发布权重

另外三项功能（多层击球描述、打法风格、脚步分析）目前还在开发分支上，尚未合并到主干。击球打分模型（`hit_scorer.json`）在本机训练，仓库里缺少该文件时自动退回手工规则。

6 结论

RallyIQ 证明了一条完全本地、免费的单目羽毛球分析流水线是可行的：深度模型只负责球和人的像素级感知，其余判断由可解释的几何规则和导出为 JSON 的小模型完成。在 ShuttleSet 留出比赛上，球场标定 89.4% 在 10 px 内，击球 F1 从 56.4% 升到 74.6%，击球方 95.4% 判对，球种 63.0%。下一阶段的关键不在算法结构，而在数据：用自拍的业余视频持续标注，通过“只有变好才替换”的闭环把转播画面上的精度迁移到用户真正拍摄的场景。

参考文献

1. Wang W.-Y. et al. ShuttleSet: A Human-Annotated Stroke-Level Singles Dataset for Badminton Tactical Analysis. KDD 2023.
2. Chen Y.-J., Wang Y.-S. TrackNetV3: Enhancing ShuttleCock Tracking with Augmentations and Trajectory Rectification. ACM MM Asia 2023.
3. Sun N.-E. et al. TrackNetV2: Efficient Shuttlecock Tracking Network. ICPAI 2020.
4. Lu P. et al. RTMO: Towards High-Performance One-Stage Real-Time Multi-Person Pose Estimation. CVPR 2024.
5. Liu P., Wang J.-H. MonoTrack: Shuttle Trajectory Reconstruction from Monocular Badminton Video. CVPR Workshops 2022.
6. BST: Badminton Stroke-type Transformer. CVPR Workshops 2026.
7. CoachAI-Projects（ShuttleNet、DyMF、RallyNet），及 IJCAI 2023 CoachAI Badminton Challenge。
8. Lin T.-Y. et al. Microsoft COCO: Common Objects in Context. ECCV 2014（COCO-17 关键点定义）。
9. Pedregosa F. et al. Scikit-learn: Machine Learning in Python. JMLR 2011（HistGradientBoostingClassifier）。

代码：yangzh0728-lgtm/rallyiq，主干 2382d53；实测记录见项目 runs/ 目录（2026-11-06 至 11-07）。